

企业级 Harness 工程实践

AI 研发、运营、数据分析

李佳奇

去哪儿旅行基础架构负责人 技术总监

AiCon

全球人工智能开发与应用大会

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

InfoQ 技术大会

QCon

上海站

全球软件开发大会

大模型正在重新定义软件

Large Language Model Is Redefining
The Software

10月22-24日 / 上海建工浦江皇冠假日酒店

· AI Agent

· Vibe Coding

· 智能可观测

· 推理基建

· 模型攻防

· AI x 创造力

InfoQ 技术大会

AiCon

北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

12月18-19日 / 北京

· 大模型架构创新

· 多模态 AI 产业融合

· 具身智能

· AI for Science

· 大模型安全

个人介绍



李佳奇

去哪儿旅行基础架构负责人，技术总监，技术中心TC委员、AI研发SIG&业务架构SIG负责人。10余年OTA一线研发经验，在高并发高可用系统建设、DDD项目落地、业务域系统价值度量、线上系统防腐治理等领域有代表性作品。2025年起全面负责AI在公司落地，在AI基础设施、AI研发、AI运营、AI数据等方向均有重量级落地成功案例。在QCon、QECon、SDCon、A2M、AiDD、GIAC、WOT、SACC、GOPS、msup TOP100峰会等大会担任出品人、讲师和专场沙龙分享嘉宾，和AZone架构院、K+Talk、musp、dbaplus等合作推出线上课程和直播。

目录

01 企业 AI 落地背景

02 AI 研发

03 AI 运营

04 AI 数据分析

05 总结

01 企业 AI 落地背景

运营、数据、研发、办公 AI 落地

去哪儿 AI 落地的几类场景



商家服务
客服服务
日常运营
社媒运营
用户运营

AI 运营



AI 智能问数
AI 商分
AI 业务波动分析
AI 经营分析

AI 数据



AI 开发
AI 测试
AI 运维

AI 研发



企业龙虾 QPaw

AI 办公

去哪儿 AI 整体架构

整体架构

入口

飞书

Web UI

API/CLI

平台

工作流 QMoss

智能问数 九章

数据分析 QNova

AI 研发 天弦

个人助手 QPaw

垂类平台

基建

LLM网关

MCP网关

Skills网关

记忆平台

Sandbox调度

模型

GPT系列

Claude系列

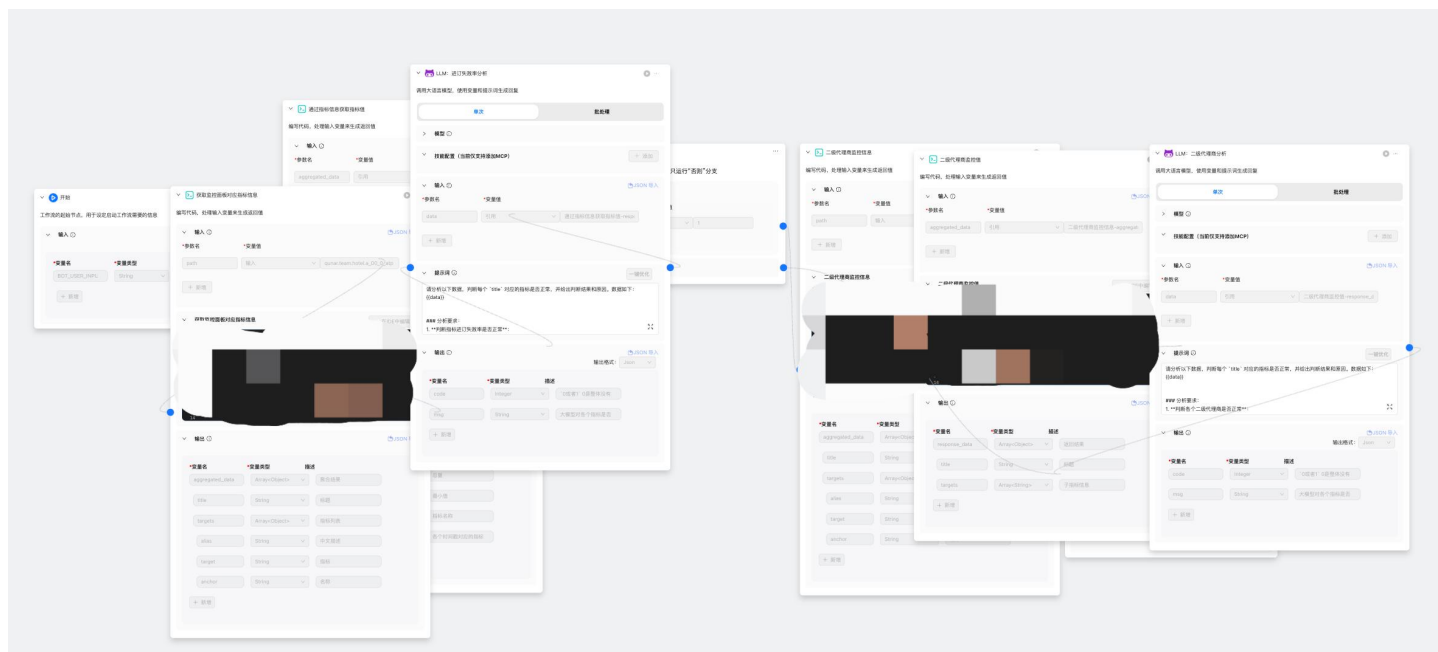
DeepSeek 系列

GLM系列

QWen系列

其它

企业级 workflow 平台-QMoss



Java 技术栈纯自研 workflow 平台

机酒火服全业务线覆盖

数万级PD提效

亿级业务收益

数据分析-QNova

QDD研发质量分析

研发质量，随问随看，报告立现

机器人

消息

云文档

+

上周基础架构的研发质量变化

(当前场景是 后端 - 研发质量分析，若非预期，请手动在底部切换场景)

正在处理您的请求，请稍后...

让我在新的任务中帮您进行分析

数据分析过程将更新到飞书文档：

数据分析过程 2026-07-17 13:20:32

(2026-07-17 13:20:32) 用户消息：

用户(richey.li)需求

上周基础架构的研发质量变化

(2026-07-17 13:20:43) AI思考：

我来分析“基础架构”团队上周的研发质量变化。首先，我需要确认团队名称、获取维度和指标列表。今天是2026年7月17日（周五），上周即7月6日（周一）至7月12日（周日）。

让我先获取相关基础信息：

运行工具：

你可阅读

数据分析报告已生成，请查看：

数据分析报告 2026-07-17 13:23:55

基础架构上周研发质量变化分析报告

分析周期：2026-07-06 ~ 2026-07-12

分析对象：基础架构（三级团队，含四级团队“基础平台”）

报告日期：2026-07-17

一、整体质量趋势洞察

停止分析

新会话

切换场景

发送给 QDD研发质量分析

数据分析报告 2026-07-17 13:23:55

QNova 数据分析 | Modified July 17

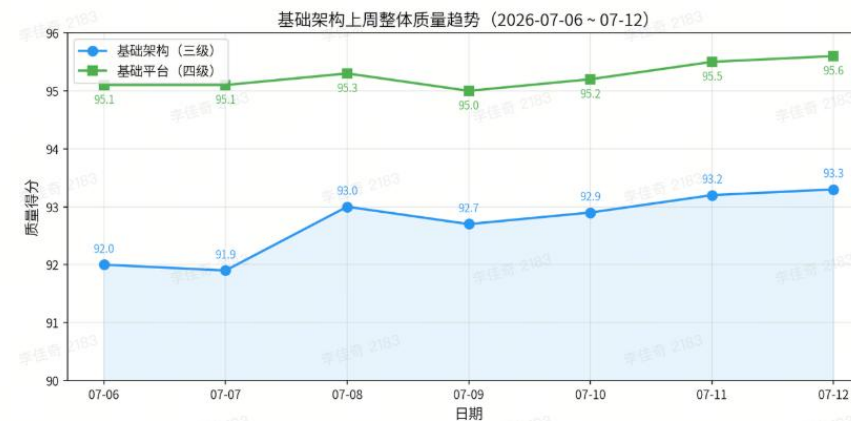
基础架构上周研发质量变化分析报告

分析周期：2026-07-06 ~ 2026-07-12

分析对象：基础架构（三级团队，含四级团队“基础平台”）

报告日期：2026-07-17

一、整体质量趋势洞察



本周期基础架构整体质量得分呈 **先降后升** 走势，周一（07-06）得分92.0，周二小幅回落至91.9（周期最低），随后持续回升至周日（07-12）的93.3（周期最高），周期均值92.7。较周初 **上升1.3分**，整体处于“达标”水平（88%~93%），周末已接近“优秀”区间（93%~98%）。

下属四级团队“基础平台”整体表现更为稳定，周期内得分从95.1稳步上升至95.6，均值95.2，始终保持在“优秀”区间。

团队指数维度（逾期故障改进计划、线上BUG率、团队项目回滚率）全周保持满分100，无波动。

二、团队表现

2.1 团队数据总览表

团队名称	周期最低分	周期最高分	周期均值	涨跌幅	主要影响指标
基础架构（三级）	91.9	93.3	92.7	+1.3	提测/发布慢查询 跳过程（周期内）

商分/数分日常工具

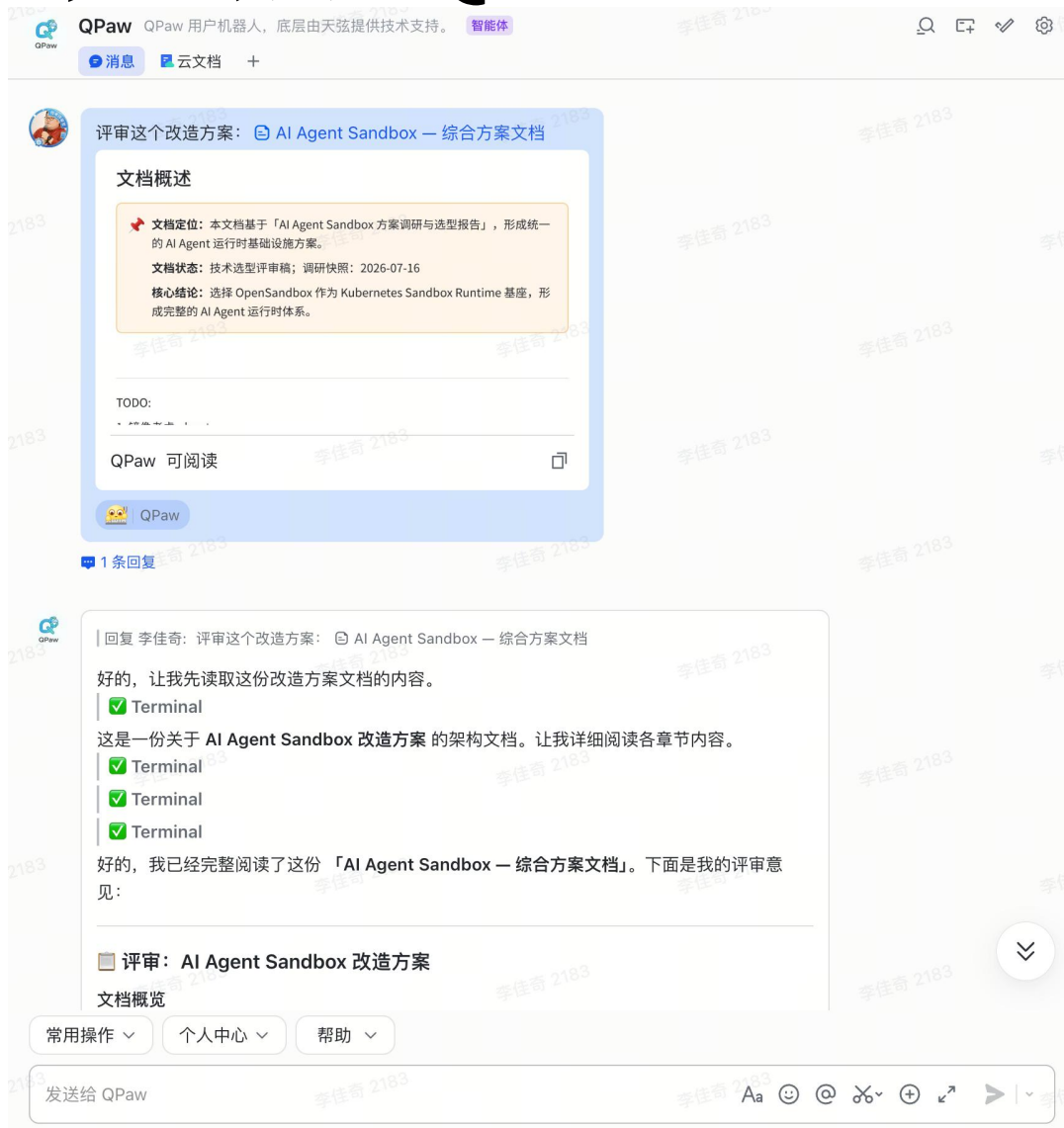
机酒火数十垂直场景

全流程数据风控

一号位经营分析



个人助手-QPaw



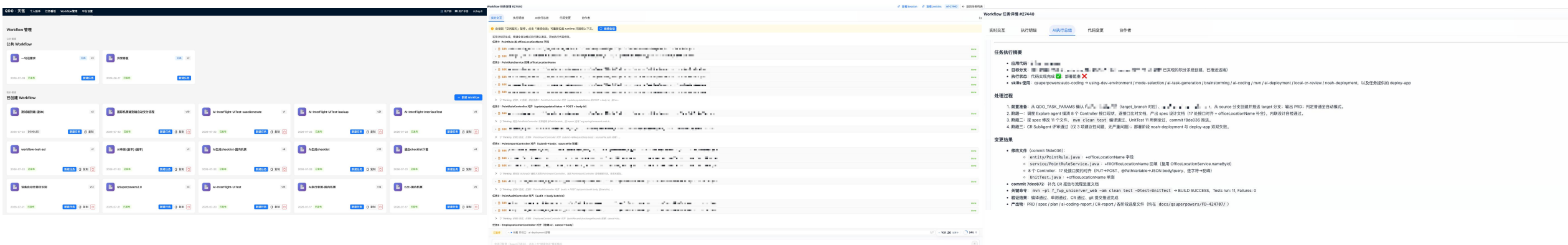
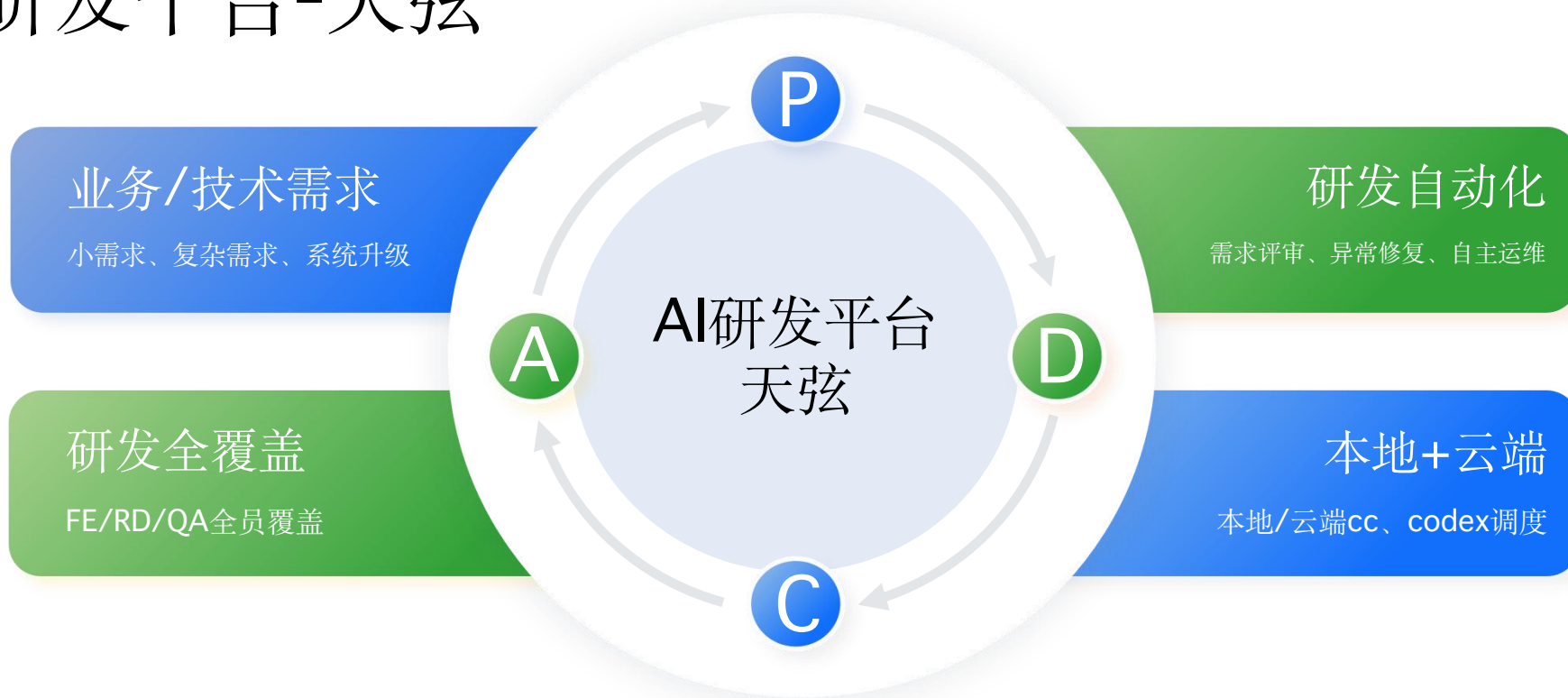
开箱即用

飞书体系深度融合

覆盖数百核心产运

企业级安全风控

AI 研发平台-天弦



02 AI 研发

天弦研发自动化平台



AI 研发平台-天弦

为什么天弦 AI 研发需要 Harness

从“模型写代码”到“平台交付任务”，中间缺的是工程化能力

核心判断

AI 研发不是生成代码，而是完成一次可验证的研发交付

企业级研发现场里，生成代码只是起点。真正的交付要穿过 需求、代码、编译、提交、部署、日志、平台回写 等真实工程环节，并留下能够判断成功与失败的证据。

天弦 Harness:

把 AI 从一次模型生成，变成一个 有流程、有状态、有证据、有边界 的研发交付系统。

一次 AI 研发任务真正要走完的工程链路

生成代码只是其中一个节点



01 流程

AI 怎么进入真实研发流程?

痛点: AI 往往停留在 IDE 或对话中,无法进入仓库、编译、部署、日志和平台状态等真实链路。

Harness: 把研发任务拆成可执行环节,让 AI 真正进入任务生命周期。

02 执行

AI 怎么稳定执行复杂系统动作?

痛点: 参数、状态、日志和工具调用如果依赖模型临时发挥,任务就难以复现和持续运行。

Harness: 把确定性动作交给脚本、状态机和工程工具,模型负责分析与决策。

03 交付

AI 怎么证明任务真的完成?

痛点: 模型说“修好了”并不代表代码真的可用、已部署、状态已回写。

Harness: 用编译、Git、部署、日志和平台状态共同形成工程证据。

核心变化: 从模型能力走向研发交付能力

真实流程

AI 进入研发现场,而不是停留在代码生成。

稳定执行

复杂工程动作由确定性机制承接。

可验证交付

用工程证据而不是模型话术证明完成。



AI 研发平台-天弦

流程编排 Harness：让 AI 研发任务真正可执行

把任务输入、计划、Agent 执行、基础能力 Skills 和研发环境组织成一条可控闭环

核心设计：AI 不直接“自由发挥”，而是在编排框架里执行

从需求任务进入开始，Harness 负责把任务拆成可执行阶段，统一连接 AI Coding Agent、代码仓库、环境配置、编译工具链、Noah 环境和基础能力 Skills，并在失败时把执行重新拉回修复环。

任务 → 编排 → 执行 → 验证

Workflow Orchestration

编排

任务编排层

决定“先做什么、后做什么、失败怎么办”

需求理解

理解任务目标、约束和验收要求，形成可执行的任务背景。

计划任务

把复杂任务拆成需求理解、代码修改、规范检查、提交、编译、测试等执行阶段。

执行顺序控制

Harness 固定流程节点，不让模型自行跳过关键验证环节。

失败回环

编译或测试失败后回到代码修改 / 规范检查，而不是直接结束任务。

结果输出

最终输出代码产物、验证结果和任务完成状态。

Runtime Execution

AI Coding Agent 是执行引擎，不是流程控制器

任务输入 INPUT

需求任务
任务描述、上下文、验收要求。

任务扩展 Skills
根据任务类型挂载对应能力。

执行约束
目标仓库、分支、环境和规则。

输出：结构化任务上下文

AI Coding Agent EXECUTE

代码理解
读取仓库和上下文。

代码修改
生成并应用代码变更。

工具调用
调用 Skills 完成确定性动作。

反馈循环
根据编译 / 测试结果继续修复。

输出：代码变更 + 执行状态

工程环境 VERIFY

代码仓库
真实 Git 工作区和目标分支。

环境配置
任务所需运行时和依赖。

编译工具链
真实编译、测试和验证环境。

Noah 环境
部署、状态检测、日志分析等能力。

输出：编译 / 测试 / 环境验证证据

Skills

基础能力 Skills

把确定性工程动作交给标准能力模块

Noah 部署

负责部署触发、环境调用和结果获取，避免 Agent 自己拼复杂接口。

启动状态检测

轮询应用启动和运行状态，为 Agent 提供确定的成功 / 失败反馈。

日志分析

统一获取应用日志、部署日志和运行日志，形成后续推理证据。

配置获取

按任务上下文获取环境和应用配置，减少模型猜测。

灭霸执行

承接特定的确定性工程动作，让 Agent 专注于任务理解与决策。

流程的价值：把 AI 放进“正确的位置”

AI Agent 负责理解任务、修改代码和做决策；Harness 负责控制顺序、边界和验证。因此 Agent 可以保持灵活，而研发流程不会失控。

最关键的设计：失败不是终点

规范检查、编译、测试形成反馈回路。失败后重新进入代码修改，而不是把一个“失败任务”直接交还给人工。

最终结果：流程可复用

同一套编排框架可以复用于 JDK 升级、一句话需求、异常修复、性能优化等场景，只替换任务和 Skills。



AI 研发平台-天弦

把 AI 的高频反馈循环变成工程基础设施

代码修改 → 编译 → Noah 测试 → 日志分析 → 再修改: Harness 决定这个 Loop 能不能高速稳定运行

● AI 研发 Loop · 可持续运行

研发验证 Loop

AI 负责推理与修复, Harness 负责让反馈快速、可控、可解释



Harness Engineering

四个机制, 决定 Loop 能不能高速运行

01 让 AI 等得起
环境创建、代码拉取、热部署、状态检测尽可能自动完成, 让每轮反馈速度匹配 AI。

02 让 AI 看得懂
专门 Java Agent 获取日志, 严格区分 Beta / Prod, 后续检索、切片和分析统一进入本地 sandbox。

03 让 AI 知道下一步
CLI 提供统一、明确、机器可解析的错误码, 让 Agent 判断继续 Loop、重试还是停止人工介入。

04 让 Loop 可治理
每一轮修改、验证和反馈都有明确边界与证据, 失败可定位, 结果可复盘。

真正的瓶颈不是模型速度

如果环境、部署、日志和错误处理比 AI 慢, AI Coding 的自动化能力最终会被底层基础设施自身吞掉。

核心判断

AI Coding 要规模化, 关键不是把模型再“催快一点”, 而是让 每一轮修改 → 验证 → 反馈 都足够快、足够安全、足够可解释。



AI 研发平台-天弦

执行 Harness：让 AI 动作稳定、可复现

把不稳定的模型行为，包进稳定的工程执行框架

● 参数 · 状态 · 证据

执行 Harness 的核心不是“增加工具”，而是“收敛不确定性”

AI 在真实研发系统里做事时，最容易失控的不是模型不会推理，而是 **参数不确定、状态不确定、证据不确定**。所以 Harness 要把这三类不确定性收敛成可复现的工程行为。

确定的交给 Harness

不确定的才交给 AI

	01 参数确定 让任务可以复现	02 状态确定 让任务知道何时结束	03 证据确定 让模型拿到正确上下文
AI 的体感	“不要让我自己猜 API、命令和参数。” 输入越稳定，AI 越少把能力浪费在参数推理上。	“告诉我任务到底成功还是失败，以及失败在哪里。” 异步工程动作必须把等待和终态判断交给 Harness。	“不要把更多日志塞给我，把真正有用的证据给我。” 上下文不是越多越好，而是要结构化、可索引、可继续推理。
问题	AI 临时拼接导致不可复现 多环境、多参数、多系统接口容易让模型临时猜测调用方式。一次调用能跑，不代表下一次还能稳定跑。	异步系统让 AI 难以判断终态 编译、部署、环境创建都有运行中、成功、失败、终止等状态。如果只返回“已触发”，AI 无法继续可靠决策。	大日志让上下文变成噪声 Jenkins、Portal、机器人和 Agent 日志体量巨大。直接输入模型容易造成上下文膨胀和关键证据遗漏。
Harness 设计	统一参数契约 由命令构建器统一生成任务类型、仓库、分支、应用、环境、Agent、callback 等参数；多环境减少必须输入项，优先自动探测和默认值。	状态机 + Wait 轮询 将 running / success / terminated / failed 收敛成稳定状态码；脚本负责等待终态，AI 只消费明确结果。	Summary + Mapping + Result JSON 先给 AI 摘要与索引，再按需读取原始证据；执行结果结构化落盘，让后续步骤能够恢复上下文、继续推理。

真正的职责分工

确定性动作由 Harness 执行，不确定性判断才交给 AI 推理

脚本负责 API、轮询、参数、文件和状态；AI 负责理解问题、选择策略、分析证据和生成修复。



AI 研发平台-天弦

AI Coding Harness · 上下文管理

上下文是分层操作系统：地图常驻、任务注入、大块走磁盘

窗口填充率当性能预算。常驻层只留 Index；工作集由控制器精确注入；大对象和审查包不进协调者 prompt。压缩时必须把入口技能和 pmo_id / appCode 灌回去。

Context Window 不是存储空间，而是受预算约束的运行时工作集

Window Fill = Performance Budget 精确装填

谁负责装填 / 裁剪

HOOK

SessionStart / PreCompact
同一脚本负责首次注入与压缩后恢复。

CONTROLLER

读取计划，只注入当前任务
overview + task-N 由控制器读取并内联。

RUNTIME

进模型前主动裁剪
deploy marker / resume 决定哪些内容直接删除。

CONTEXT WINDOW

模型只看到“此刻必须知道”的内容

谁真正消费上下文

L1

PINNED MAP

常驻地图：只保留入口技能 + 会话级主键

using-qsuperpowers 是 Index；其它技能通过 Skill() 按需加载。Compaction 后重新灌入入口技能与任务身份。

using-qsuperpowers pmo_id appCode Skill() on demand

L2

ACTIVE WORKING SET

当前工作集：一任务一上下文，由控制器精准注入

implementer 只拿任务全文 + 必要依赖；reviewer 只拿路径。子代理禁止自己翻计划目录，避免把 Graph、历史任务和旧报告带进窗口。

task-N 全文 依赖 / 架构上下文 BRIEF_FILE REPORT_FILE DIFF_FILE

COORDINATOR

只持有索引与调度状态
不承载大报告、diff 包和完整 CR prompt。

IMPLEMENTER

新 subagent = 新工作集
只收到当前任务刚好够用的信息。

REVIEWER

路径驱动审查
审查包留在磁盘，通过路径进入 reviewer。

L3 外存

DISK AS CONTEXT BUS

\$EXEC_DIR/subtask-~-impl-report.md spec-report.md quality-report.md prompt_file review-package \$SKILL_DIR/agents/*

默认留在窗口外

大报告 / diff / CR prompt / 未选中的部署段 / 过期需求，需要时传路径：进入模型前先裁剪。

L1 Hook 钉地图：压缩后重新灌入

- SessionStart / PreCompact 协同一脚本
- <SUBAGENT-STOP> 让子代理跳过地图
- additionalContext 只注入入口 SKILL
- pmo_id / appCode 禁止 compaction 丢失

```
session_context = {
    "仅常驻 using-qsuperpowers: 其它技能走 Skill()\n"
    f"{skill_path.read_text()}"
}
output = {"additionalContext": session_context}
```

L2 Controller 装工作集：subagent 不自己翻计划

- 计划任务全文由控制器读取并粘贴
- 修复仍漏 implementer，控制器不亲手改
- 每个任务创建新 subagent 隔离上下文
- 技能资源统一通过 \$SKILL_DIR 解析

```
## 任务描述
[计划中任务的完整文本 -
在此粘贴，不要让子代理读取文件]

## 上下文
[任务位置 / 依赖 / 架构上下文]
```

L3 磁盘做总线：传路径，进模型前再裁剪

- \$EXEC_DIR 存中间报告与 manifest
- prompt_marker 按 deploy_env 删除无关段
- CR prompt 落临时文件，只返回 prompt_file
- Resume 时 rendered_prompt = ""

```
# 大 prompt 不进入协调者上下文
temp_file.write_text(full_prompt)
result = {"prompt_file": str(temp_file)}

# 模型不需返回 requirement
if skip_initial_prompt:
    rendered_prompt = ""
```

01 谁有权出现在窗口里
Hook 只钉入口技能；子代理禁用地图

02 谁负责装填工作集
控制器精确注入；subagent 不自己读计划

03 谁被故意留在窗口外
报告、diff、CR prompt、未选部署段、过期需求

CONTEXT / 01



AI 研发平台-天弦

AI Coding Harness · Skills 设计

先让模型拿到对的作业指导书，再允许它动手

Skill 不是把规范写得更长。它是一张必须先打开才能干活的作业指导书：封面决定开哪本，开头有红灯，细节做到哪一章打开哪一章，开工前还要先把指导书装进工位。

一张 Skill 从“被找到”到“真正约束 Agent”的完整路径

公司不会把整面墙的 SOP 一次性贴在工位上



总控只派活，不亲自干活

auto-coding → 需求 → 规格 → 规格 → 代码 → 代码 → 环境 → 环境 → 验证

项目经理下场写代码 = 绕过质量闸门

01 **封面决定开哪本**

模型开场看不到全书，只看到封面上的一小段话。封面必须讲清“什么时候找我、什么时候别找我”，宁可多查一次目录，也不要凭记忆开干。

一句工程落地
用 description 做互斥路由：入口规则要求“哪怕 1% 可能适用，也先打开 Skill 查”。

```
# 封面：系统只靠这段话决定开不开你
description: >
什么时候用：还没有可执行技术方案，需要先设计
触发命令：“生成方案”“怎么实现”
什么时候不用：已有完整方案 —— 去用“执行方案”
```

02 **开头是红灯；细节按章翻**

第一页不是建议，而是**红灯**：条件不满足就不能继续。后面的细节做到哪一步才打开哪一章；总控技能只负责派活，不能自己越权下场。

一句工程落地
停机条件放在技能最顶部；流程拆成阶段文件逐步加载；编排器明确禁止自己写规格、编码和部署。

```
计划获得批准之前，不要调用任何实现技能，
不要写任何代码。
# 人话：施工图没签字，工具箱不准打开
本技能是编排器，不亲自做分析、编码、部署。
```

03 **指导书必须先装进工位**

写在文档库里，不等于正在写代码的 Agent 拿得到。技能必须在 Agent 开口之前安装好；装失败就整单停。指导书只说动作，具体产品按钮由映射表翻译。

一句工程落地
流水线固定为：准备 → 缓存判断 → 拉代码 → 安装 Skills → Agent 开始写；指定技能安装失败直接失败。

```
# 开口前安装
准备环境 → 判断缓存 → 拉代码
→ 安装本次 Skills
→ Agent 开始写
ruler skills install <技能名> --tool claude
配置了但安装失败 → 整单失败
```

01 **怎么被选中**
封面写清何时用、何时不用；宁可多查目录

02 **凭什么停手**
开头是红灯；细节按章翻；项目经理不许下场

03 **怎么真的用上**
开口前装进当前 Agent；动作和产品按钮各分开



AI 研发平台-天弦

需求 Harness：从原始需求到高质量 PRD

把需求理解、规范校验、冲突识别和知识沉淀固化成可重复的 AI 需求处理流程

需求处理的核心目标

AI 不只是“帮忙写 PRD”，而是把分散的需求输入转化为 **标准化、可执行、可评审、可追溯** 的产品文档，并在过程中沉淀业务知识。

输入 → 校验 → 推理 → 交付

需求 Harness

需求 Harness 核心链路

01 多源输入

统一收集需求上下文

输入来源

- ✓ 需求规范
- ✓ 产品文档
- ✓ 历史知识库
- ✓ 用户补充说明

HARNESS 产物

结构化需求上下文

明确业务目标、用户对象、范围、约束和已有背景，减少信息遗漏。

02 规范校验

先判断需求是否“说清楚”

自动检查

- ✓ 文档标准化校验
- ✓ 不合理 / 冲突识别
- ✓ 场景补充与完备性检查
- ✓ 关键条件缺失检测

HARNESS 产物

问题清单 + 补充建议

在进入 PRD 生成前把歧义、冲突和缺失项暴露出来，而不是把问题留到评审阶段。

03 AI 推理增强

把需求从“描述”变成“可执行方案”

智能处理

- ✓ 补齐业务场景
- ✓ 关联历史知识与案例
- ✓ 识别需求边界和依赖
- ✓ 量化评分分析与可视化建议

HARNESS 产物

可执行需求结构

从“我要什么”进一步形成“怎么定义、怎么分析、需要什么数据和依赖”。

04 PRD 交付

形成可评审、可追溯的最终文档

交付标准

- ✓ 全面：覆盖关键业务场景
- ✓ 清晰：结构与表达统一
- ✓ 可测：需求可验证
- ✓ 可追溯：知识与决策有来源

最终产物

高质量 PRD

不仅交付文档，同时把审核规则、问题模式和业务知识沉淀回知识库。

从“写 PRD”到“管需求质量”

Harness 的价值不只是提高文档生成速度，更重要的是把 **需求质量控制前移**，在需求进入研发前发现歧义、冲突和缺失。

把标准变成可执行规则

文档规范、场景完备性、评分与可视化等要求，不再依赖个人经验，而是沉淀为 **可重复执行的检查机制**。

把一次处理变成持续学习

每次需求处理产生的问题、建议和知识都可以继续沉淀，形成 **需求知识库 → 需求处理 → 反馈 → 知识增强** 的闭环。



AI 研发平台-天弦

AI 测试用例生成 Harness

不是生成更多 Case，而是把需求变化映射成一套可解释、可复用的测试空间

Test Case Generation

06

不是“写几个 Case”，而是先把测试空间算清楚

先识别功能边界，再决定测试类型、覆盖方式和生成规则。

需求理解 → 路由 → 生成

测试空间计算

输入

PRD 文档

业务目标、功能边界、场景和验收要求。

需求语义

技术方案 / AI Task

实现方式、关键链路、配置约束和任务目标。

技术上下文

代码改动 Diff

本次变更影响的模块、接口、逻辑和关键代码。

变更证据

输入不是越多越好

Harness 先做结构化，再把真正有用的信息送入测试空间计算。

处理层

测试空间计算器

把“需求变化”拆解成测试对象、测试策略和具体 Case

让测试策略可解释、可复用、可扩展

测试类型路由规则

不同需求进入不同测试策略，不再“一套 Case 打天下”

技术层

先确定基础测试方式，再叠加业务特征

接口类

- 参数校验
- 错误码 / 异常流
- 依赖 / 幂等

前端 UI / 交互

- 兼容性
- 交互体验
- 页面状态

配置开关

- 灰度 / 生效范围
- 配置异常
- 默认值

业务特征

高风险或特殊机制追加专属测试策略

资损 / 金融

- 金额计算
- 精度 / 舍入
- 特殊场景

流程状态

- 状态机覆盖
- 流转校验
- 异常分支

AB 实验

- 流量分配
- 指标埋点
- 实验设计

通用功能

- 正常 / 异常 / 边界
- 输入输出
- 单程 / 多线程

路由依据

测试设计原则

约束覆盖方式、优先级、分层规则和必测场景。

业务规则库 + 通用场景库

复用业务规则、历史场景和通用验证模式，补足隐性边界。

代码证据

结合 Diff、模块关系和关键链路判断变更影响范围。

输出

测试用例

正常、异常、边界、配置、性能、安全等多维 Case。

可执行

测试覆盖

按功能、类型和场景统计覆盖，识别漏测区域。

可衡量

执行输入

结构化 Case 直接进入后续自动化测试执行 Harness。

可衔接

下一步

测试 Case → 自动执行 → 结果分析 → 缺陷证据。

核心价值

让测试从“依赖工程师经验”变成 可计算的测试空间 + 可解释的路由规则 + 可复用的测试资产。



AI 研发平台-天弦

安全 Harness：让 AI 能自动做事，但不能失控

AI 可以进入研发系统，但每一个高影响动作都必须有明确边界、规则和兜底机制

四道边界 · 三档控制

AI 自动化的前提不是能力，而是边界

安全 Harness 不是把 AI“关住”，而是明确三档控制：什么可以自动做、什么必须受控、什么必须人工接管，并把规则落实到工具、脚本、平台和流程。

统一控制尺度

自动

受控

人工接管

01 代码边界

主要风险

误提交 · 误修改

受控

目标分支必须受控，AI 只能进入允许的工作分支。

禁止

master 直接提交、force push、删除远程分支、merge commit。

自动

代码变更自动留痕，形成可审查、可回滚的 diff 与提交记录。

人工接管：进入受保护分支或触发高风险 Git 操作时，停止自动执行。

02 系统边界

主要风险

生产 · 通知 · 外部资产

自动

查询、分析、编译等确定性动作可以自动执行。

受控

部署、外部通知、写入系统进入明确的能力和边界。

人工接管

生产发布、不可逆操作、客户影响不能默认放开。

控制手段：受控代理、sandbox、No-Op 服务和环境隔离，减少外部副作用。

03 权限边界

主要风险

越权 · 高风险参数

自动

在授权范围内调用工具、API、数据和环境。

禁止

自动补高风险参数，不能绕过审批和既有职责边界。

受控

权限不足或超出职责范围时，必须暂停并请求人工判断。

控制手段：工具白名单、参数约束、权限校验、审批状态和人工确认。

04 交付边界

主要风险

假完成 · 状态失真

自动

自动执行编译、部署、测试、状态查询并采集结果。

受控

通过callback / DB / 日志校验平台状态和真实工程结果。

禁止

无工程证据不得回写“完成”状态，证据不足必须人工接管。

控制手段：编译结果、提交记录、部署状态、日志证据共同形成交付证明。

贯穿四道边界的硬护栏原则

安全边界不能写在提示词里，必须写进 工具、脚本、平台规则和流程闭环。



AI 研发平台-天弦

项目 Harness：持续构建项目上下文，驱动项目执行

不只是管理项目任务，而是持续理解组织、项目、人员、信息和风险，并让 AI 参与项目执行

项目 Harness 的核心目标

项目运行过程中，关键信息不断变化，也不断分散在文档、沟通记录、开发测试环境和人员经验中。Harness 要持续完成 **信息补全**、**上下文理解**、**任务编排** 和 **流程适配**，让 AI 始终基于“当前项目真实状态”工作。

理解项目 → 组织执行

Project Harness

项目 Harness 核心能力链路

01

项目上下文

持续收集并补全项目真实状态

上下文构建

- ✓ 项目目标、范围与生命周期阶段
- ✓ 组织架构、角色与职责边界
- ✓ 项目文档与历史决策
- ✓ 会议 / IM / 邮件中的新信息
- ✓ 开发、测试、发布环境中的状态信息

HARNESS 产物

动态项目上下文

不仅知道“有什么资料”，还持续补入新信息，识别隐藏在项目过程中的关键事实。

02

理解与对齐

把分散信息变成团队共同认知

智能处理

- ✓ 主动发现信息缺口并发起澄清
- ✓ 从文档、沟通、环境中挖掘隐性信息
- ✓ 识别不同人员之间理解偏差
- ✓ 补充关键决策、约束与依赖
- ✓ 持续更新项目共识

HARNESS 产物

统一项目认知

让“知道的人知道、其他人不知道”的隐性信息逐步显性化，减少信息差和对齐成本。

03

任务编排

围绕项目分工推动正确的任务执行

任务判断

- ✓ 识别当前生命周期阶段和下一步目标
- ✓ 从项目状态中识别待执行任务
- ✓ 结合角色、能力和负载进行分配
- ✓ 识别前置依赖和协作关系
- ✓ 明确责任人、验收标准与截止时间

HARNESS 产物

任务编排与责任分配

不是简单自动派单，而是结合组织结构和项目实际状态，把任务交给正确角色并形成协作闭环。

04

风险与流程适配

识别风险，同时适配不同项目的运行方式

动态判断

- ✓ 进度、质量、资源与协作风险
- ✓ 需求变化和关键路径偏移
- ✓ 识别项目当前需要的管理动作
- ✓ 适配不同项目类型与生命周期
- ✓ 适配不同任务类型、团队和组织流程

HARNESS 产物

风险预警 + 流程适配

同一套 AI 能力不强行套统一流程，而是根据项目类型、任务类型和团队特点调整执行方式。

从“项目资料”到“项目事实”

Harness 不只读取 PRD、计划和会议纪要，还主动从沟通记录、开发测试环境和人员经验中提取**关键事实与隐性信息**。

从“任务列表”到“组织执行”

AI 不是单纯生成待办，而是理解**谁应该做、为什么现在做、前置依赖是什么、需要哪些人协作**。

从“固定流程”到“动态适配”

不同项目、不同任务、不同团队不应该强行走同一流程，Harness 需要具备**按场景动态适配项目运行方式**的能力。

03 AI 运营

QMoss workflow 平台



AI workflows 平台-OMoss

Qmoss Harness: 让 AI 把流程做对

业务 Workflow 下的 Context / Decision / Action / Convergence

从“AI 会做”到“AI 把流程做对”

AI 持续读取状态、在流程约束内决策、执行真实动作，并根据结果继续推进到合法终态。

业务闭环：持续推进到终态

环内只保留短句，所有解释集中到右侧



异常 / 重试 / 状态变化后回到 Decision，直到成功终态或人工终态。

四个 Harness 控制面

分别对应业务 Workflow 的状态、决策、动作与收敛

01 Context

状态驱动的上下文，而非一次 Prompt
每一轮决策都读取当前状态、历史动作与流程位置。

02 Decision

在流程约束内决策，而非自由发散
Agent 可以智能推理，但不能脱离用户配置的业务流程。

03 Action

Tool 成功 ≠ 业务成功
动作完成后必须确认真实业务状态，再决定下一步。

04 Convergence

有限时空内必须停下来
异常与重试不能让 Workflow 无限运行，必须进入合法终态。

Convergence 的两种合法出口

自动完成，或安全交给人工

SUCCESS

成功终态 · 任务完成
业务结果确认，Workflow 正常结束。

HUMAN

人工终态 · 受控接管
无法继续时，带状态与执行证据进入人工处理。

本页结论

Qmoss Harness 的核心不是让 Agent 更聪明，而是让 用户配置的业务 Workflow 可理解、可执行、可控制、可收敛。



AI workflows 平台-QMoss

Qmoss Harness：先证明 Workflow 可运行

编译期拦截结构与数据流错误，通过门禁后再进入 Runtime 解释执行

把错误挡在运行前

用户画布是声明式程序，Harness 先校验，再执行；校验失败直接阻断，不把确定性错误带进 Runtime。

编译期：四类前置检查

左边回答“拦什么”，右边直接给出失败后果

01 节点完整性

检查孤立节点、断链以及必要节点是否可达。

不通过 → 拒绝发布

02 连通性与分支

检查不可达路径、无出口分支和分支覆盖完整性。

无出口 → 直接拦截

03 数据流约束

保证上游输出可以被下游节点正确引用和消费。

断引用 → 禁止 Runtime

04 静态环检测

提前识别单图内无法退出的静态循环结构。

静态环 → 禁止上线

校验通过
进入 Runtime

Fail
阻断发布

运行期：通过后如何解释执行

编译链路只证明“可运行”，Runtime 才负责真正执行和状态回写

用户 Workflow

用户配置的节点、条件、分支与动作



合法 DAG · 已证明可运行

结构、数据流、分支和静态环均通过检查



Runtime · Interpret

按图解释执行，并在节点完成后持续回写状态

正常：状态反馈驱动续跑

节点结果进入运行态上下文，驱动后继节点继续执行。

异常：人工 / 受控接管

异常路径必须有合法出口，并进入受控人工处理。

Harness 编译期职责 先把用户 Workflow 编译成已验证的可执行图，再进入 Runtime 解释执行。

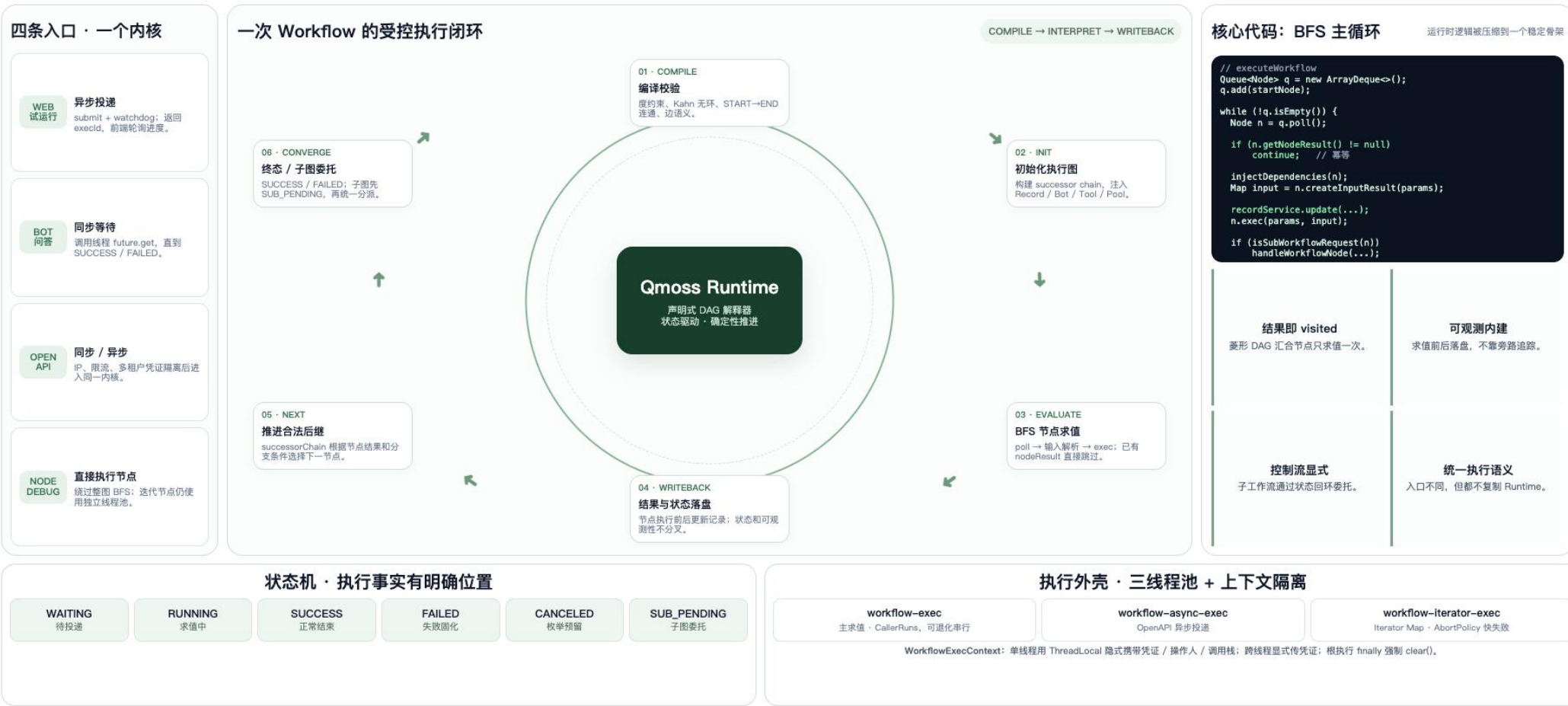


AI workflows平台-OMoss

Moss Harness · 可运行

把不可信的用户画布，编译成一次受控、幂等、可观测的 DAG 求值

所有入口先收敛到同一套执行语义，再沿“校验 → 初始化 → 求值 → 落盘 → 推进 → 终态”闭环运行；图越可信，Runtime 越简单。



可运行靠前置不变量最大化：编译器把复杂性挡在前面，Runtime 才能用最简单、最稳定的循环完成执行。



AI workflows 平台-OMoss

Qmoss Runtime Harness：让每一轮决策都基于最新业务状态

左边定义状态，中间完成状态转换，右边由执行内核保证流程可重复、可推进

运行时上下文 = 随节点更新的业务状态 不是聊天记忆，也不是一次 Prompt；每一次 Decision 都读取上一轮真实执行产生的新状态。

Context：每次决策读什么

状态模型决定 Loop 能否基于真实结果继续推进

- 01 全局业务状态**
用户目标、流程位置和节点结果随执行持续更新。
- 02 结构化结果引用**
下游节点按明确路径读取上游输出，避免上下文靠猜。
- 03 历史结果不可篡改**
已完成节点结果形成快照，后续执行不能回写历史事实。
- 04 状态 + 记录一体**
运行状态与执行记录同步沉淀，进度可观测、结果可回放。

Runtime Loop：每一轮怎么转

Action 的真实结果写回状态，再成为下一轮 Decision 的输入



执行内核：Loop 如何稳定推进

把概率性 Agent 放进确定性的 Workflow 解释器

确定性的部分由 Runtime 控制，不确定的部分才交给 AI 推理。

- 01 按依赖推进**
队列按图关系推进节点，避免 Agent 自由跳过流程步骤。
- 02 已完成节点不重复执行**
节点结果已存在时直接复用，避免依赖汇聚导致重复求值。
- 03 不同节点统一调度**
模型、工具、条件和业务动作都进入同一套执行骨架。
- 04 数据边界在内核受控**
引用、类型转换和深度等约束由 Runtime 统一保证。

本页结论

运行时上下文不是聊天记忆，而是 随节点执行持续更新、可被下一轮决策直接消费的业务状态。



AI workflows 平台-OMoss

Qmoss Runtime Harness: 让 Workflow 最终可停

通过时间、空间与失败语义边界, 把 AI 的异常限制在可控范围内

AI 不必每一步都正确, 但不能无限运行

Workflow / Loop / Iterator 统一进入 Runtime; 默认护栏可配置, 用来限制循环、并行与资源扩张。

≤100

循环次数

≤100

迭代次数

≤10

并行任务

≤5

参数深度

有限执行空间: 把“跑飞”变成可控边界

控制时间、循环、并发与僵死状态, 确保 Runtime 不无限消耗资源

01

动态环检测

跨 Workflow 的动态调用环进入执行栈时直接识别并阻断。

02

双层超时熔断

整图与单次迭代分别设置超时, 避免单节点拖死全局。

03

多租户资源限流

按调用方隔离 Token 与并发资源, 避免单租户挤占系统容量。

04

僵死状态主动归位

扫描长期 RUNNING 任务, 主动结束异常状态并触发告警。

失败语义: 失败也必须成为可信状态

运行失败不是例外, 失败结果本身也必须可观测、可判断、可继续处理

语义不吞 · 边界不越 · 状态可信 · 最终可停

01

错误分层: 用户可见 vs 系统内隐

用户错误提供可读反馈; 系统异常脱敏并进入内部告警链路。

02

失败原子: 节点必须进入 FAILED

失败节点先固化状态, 再中断流程, 避免长期挂在 RUNNING。

03

先固化再传播: 记录与真实状态一致

失败不会被吞掉, 执行记录始终反映真实运行结果。

三种合法停法

结束 ≠ 只有成功一种结果

SUCCESS

成功终态 · 任务完成

业务结果已确认, Workflow 正常结束。

FAILED

失败终态 · 错误已固化

错误可观测、可追踪, 后续是否重试由策略决定。

HUMAN

人工终态 · 受控接管

AI 无法继续时, 带着状态与执行证据进入人工处理。

本页结论

收敛的终点不是“每一步都成功”, 而是 Workflow 最终进入成功、失败或人工中的一个可信终态。



AI workflows 平台-QMoss

Moss Harness · 可终止

每条执行路径都有停止条件；停止后，状态先固化再传播

终止不是一个按钮，而是一套纵深机制：运行前能拒绝，运行中能熔断，失败时能固化，僵死后能恢复；每个设计点都由代码边界保证。

终止控制系统：Guard + 状态机 + 超时 + 恢复

01 ADMISSION → 04 RUN → 05 TERMINATE → 06 FAILED → 08 RECOVER

前置 Guard：准入 / 结构 / 资源

ADMISSION GUARD

准入拒绝

Token 限流，IP 白名单，图校验失败直接不投递。

```
if (tokenRateLimiter
    .tryAcquire(token))
    throw new MossSystemException(
        "rate limit", ...);
```

STRUCTURE GUARD

静态环 + 动态调用环

Kahn 拉单面环：执行栈再跨面 A→B→A。

```
if (executionStack
    .contains(workFlowId))
    throw new WorkflowInvocationException(
        "存在循环依赖");
```

RESOURCE GUARD

资源边界写进节点

Loop≤100，Iterator≤100，并行≤10，输出超限主动失败。

```
iteratorTime =
    Math.min(iteratorTime, 10);
execMaxTime =
    Math.min(execMaxTime, 100);
```

Execution 生命周期：终止必须留下可信终态



运行 Guard：时间 / 异常 / 流控

TIME GUARD

时间熔断

整型和 Iterator 子任务都通过 Future.get(timeout) 控制。

```
future.get(timeout,
    TimeUnit.MINUTES);
...
future.cancel(true);
```

ERROR GUARD

失败先固化再传播

异常不吞：finally 无论成败都更新执行记录。

```
catch (Exception e) {
    preNode.endNodeExec(
        null, FAILED, e.getMessage());
    throw e;
}
```

FLOW / BACKPRESSURE GUARD

主动退出 / 池满快失败

Break 写 LOOP_FLAG=false；迭代池 AbortPolicy 拒绝即 FAILED。

```
ctx.put(LUMP_FLAG, false);
return false;
endNodeExec(
    null, FAILED, e.getMessage());
```

生产级终止设计的三个工程亮点

ENGINEERING 01

失败先固化，再传播异常

节点异常时先写入 FAILED，finally 无论成败都更新执行记录，再向上抛异常。状态与真实执行始终一致。

ENGINEERING 02

幂等结果天然支持恢复

nodeResult 同时承担 visited，执行进度和恢复断点。恢复时已有结果直接 skip，只继续未完成部分。

ENGINEERING 03

边界直接写进 Runtime

超时、Loop 次数、Iterator 并行度、参数深度、输出体积和 Token 限流都有明确硬边界，不依赖模型自律。

双层超时 · checkTimeout 是终止的关键落点

submit

任务进入执行线程池

get(timeout)

等待正常结束

TimeoutException

先记录 FAILED + 超时原因

WEB

主执行 + 旁路 watchdog

cancel(true)

最后 interrupt 工作线程

```
// WorkflowExecService.checkTimeout
try {
    future.get(timeout, TimeUnit.MINUTES);
} catch (TimeoutException |
    InterruptedException |
    ExecutionException e) {
    runRecord.setExecStatus(
        ExecStatus.FAILED);
    runRecord.setErrorMessage(
        "工作流运行超时");
} finally {
    workflowRunRecordService
        .updateRunRecord(runRecord);
    future.cancel(true);
    // 清除线程自己 future.get
    // 每个子 Future 再查一遍
```

僵死恢复 · nodeResult 既是结果，也是断点

扫描

找超时 WAITING / RUNNING

识别断点

已有 nodeResult → skip

复用 BFS

从未完成节点继续执行

重新固化

SUCCESS 或再次 FAILED

```
// workflowRecoverExec
if (temp == null ||
    temp.getNodeResult() == null) {
    preNode.exec(...);
} else if (
    temp.getNodeResult() != null) {
    .getErrorMessage() != null) {
        throw new CheckedException(
            temp.getNodeResult()
                .getErrorMessage());
    }
    // else: 已有 nodeResult → skip
```

```
// 失败语义：先固化，再传播
try {
    preNode.exec(params, input);
} catch (Exception e) {
    preNode.endNodeExec(
        null, FAILED, e.getMessage());
    throw e;
} finally {
    recordService.updateNodeResultsByExecId(...);
}
```

可运行靠前置不变量；可终止靠纵深防御。失败先固化后传播；终止能力通过状态固化、超时熔断、资源边界与幂等恢复形成完整生产闭环。

04 AI 数据分析

QNova



AI 数据分析-ONova

QNova Harness：把开放式分析装进企业级 Runtime

从 LLM 能力调用到可治理、可恢复、可交付的数据分析任务

企业数据分析不是一次问答，而是 生产任务

QNova 要把开放式推理、企业数据边界和长任务交付装进同一个 Runtime Harness。

三类核心矛盾

左边讲“为什么需要 Harness”，并直接对应右侧分析链路

开

分析开放性

用户问题不固定，Agent 需要自主拆解、查询、计算和归因。

理解 / 分析

边

数据约束性

表、字段、数据量、写操作和企业上下文都必须受控。

数据 / 执行

产

任务生产性

分析链路长，需要状态恢复、实时反馈、终止和最终报告交付。

报告 / 交付

一次真实分析链路

六步形成一条完整生产链路，左侧三类矛盾分别在不同阶段暴露

01 理解

问题理解与拆解

把开放问题转成可执行分析计划。

02 数据

查元数据 / 选数据源

决定能看什么、能查什么。

03 执行

SQL / Python / MCP

模型选择工具，Harness 管理边界。

04 分析

计算 / 归因 / 再查询

多轮分析持续产生中间结果。

05 报告

图表 / 结论 / 报告

结果不能只停在模型输出。

06 交付

Web / 飞书 / OSS

最终产物进入真实协作环境。

本页结论

QNova Harness 的核心，是把一次开放式数据分析任务变成 有场景、有状态、有边界、有交付 的生产任务。



AI 数据分析-ONova

QNova Harness：四个控制面

Scene 定环境 · Runtime 管任务 · Governance 管边界 · Delivery 管交付

四个控制面协同工作

四个控制面共同约束一次复杂的数据分析任务，从运行前配置一直覆盖到最终交付。

01 Scene

在哪个业务分析环境里工作？

业务知识	场景描述、推荐问法、分析口径。
数据边界	文件 / Hive / StarRocks / 飞书 / MCP。
工具集合	Python / SQL / 报告 / MCP 按 Scene 装配。
执行策略	模型、并行、超时、子 Agent 等约束。

Scene ≠ Prompt，而是分析应用描述语言

02 Runtime

分析任务如何长期、稳定地运行？

外层任务图	prepare → analyze → end，控制任务生命周期。
内层 Agent 图	LLM → Tool → Result → Continue，开放式分析。
Checkpoint	主任务状态持久化，失败后可继续。
Stream / Poll	长任务实时反馈，也保证最终可获取。

模型决定怎么分析，Harness 决定任务怎么活着跑完

03 Governance

AI 可以分析什么，又不能碰什么？

Prompt Rule	先查元数据、字段来自元数据、禁止写操作。
Tool Enforcement	执行前强校验表、字段、数据量和风险行为。
Runtime Context	User / Task / Scene / Entry 由框架注入。
安全边界	DDL / 写操作禁止，超限进入治理链路。

Prompt 告诉 AI 应该怎么做，Tool 保证只能这么做

04 Delivery

分析如何发散，又最终收回来？

Multi-Agent	父 Agent 拆分，子 Agent 独立分析。
结果归并	子结果流式回传，父 Agent 统一综合。
上下文预算	压缩、截断、OSS 大结果外置。
Output Fence	飞书 / Web 交付前做最终质量处理。

Agent 可以发散，Harness 必须收回结果并交付

本页结论

四个控制面共同约束一次复杂分析任务，形成完整的分析 Runtime。



AI 数据分析-ONova

QNova Harness: Scene + Runtime

Scene 定义分析环境，Runtime 让开放式分析任务长期、稳定、可恢复地运行

Scene as Application: 把分析 Agent 配置成应用

Scene: 运行前定义分析环境

不是给 Agent 补一点上下文，而是定义一个完整分析应用

Knowledge

业务知识 & 推荐问题
沉淀分析口径、常见问题和业务解释上下文。

Data

数据源与可访问范围
文件、Hive、StarRocks、飞书表格和 MCP 服务。

Tools

工具按 Scene 装配
Python、SQL、节假日、上传文件、MCP 等工具动态挂载。

Agent

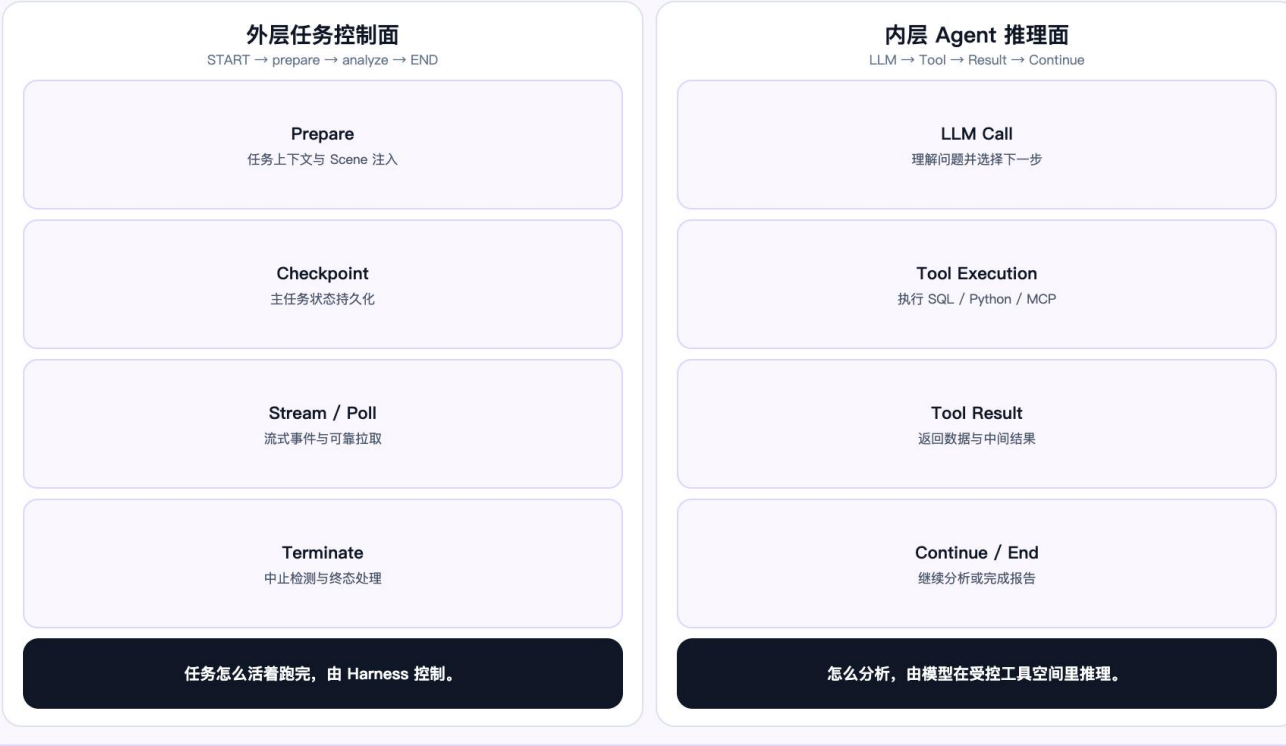
Agent 模式与并发策略
是否启用多 Agent、子任务超时、最大并行数等。

Model

模型策略与历史压缩
按场景选择模型，并控制历史代码和长上下文压力。

Runtime: 双层 Graph 分离控制面与推理面

任务控制权留给工程系统，分析决策权交给模型



本页结论

QNova 的第一性设计是 任务控制权留给工程系统，分析决策权交给模型。



AI 数据分析-ONova

QNova Harness: Data Governance + Runtime Context

企业数据分析 Agent 必须把访问边界下沉到 Tool Runtime，把关键企业上下文注入执行过程

Data Governance: AI 可以分析，但不能自由访问数据

为什么数据访问最危险

模型会写 SQL，不代表模型应该能访问任意数据

不能访问任意表

表必须在 Scene 配置白名单内，不能自由探索企业数据资产。

不能访问任意字段

SQL 字段需要经过审批，字段级权限不能只靠 Prompt。

不能返回任意数据量

结果量受到审批限制，超限需要事件流和管理员提醒。

不能执行写操作

DDL 和写操作必须禁止，只允许在分析边界内查询。

只有 Prompt

模型可能不遵守，边界缺乏最终保障。

双层防线: Prompt Rule + Tool Enforcement

不是二选一，而是“引导 + 强制”共同保证

Prompt

告诉 AI 应该怎么访问

先查元数据、字段来自元数据、禁止写操作、只用 Scene 数据源。

Tool

执行前做强校验

真实执行前检查白名单、审批字段、查询规模和风险行为。

Metadata

先知道“有什么”，再写 SQL

减少模型编造表名、字段名和数据结构。

Whitelist

只能查询 Scene 配置的数据

Hive / StarRocks 使用范围由场景配置决定。

Approval

字段与数据量强校验

QA 审批与数据量限制在 Tool Runtime 真正执行。

Alert

超限进入治理链路

通过事件流和管理员通知暴露异常访问。

只有 Tool

模型不断试错，体验和效率都会下降。

Runtime Context: 关键上下文不让模型传

模型只负责业务参数，框架在工具执行时注入企业运行态

User / Task

当前用户与任务 ID 由 Runtime 注入，不信任模型自己携带。

Scene / Entry

当前 Scene、入口 Web / 飞书等运行元数据自动绑定。

Files / MCP

上传文件归属、MCP Client 生命周期由任务运行态管理。

Report Target

当前报告位置由框架传给工具，避免模型拼接关键路径。

两层结合

模型更容易做对，系统又能在执行点真正拦截。

本页结论

QNova 不信任模型一定遵守规则，因此把治理从 Prompt 下沉到 真实 Tool Runtime。



AI 数据分析-QNova

QNova Harness：让分析发散，但必须收回结果

Multi-Agent 负责拆分与探索，Production Delivery 负责恢复、终止、预算和最终交付

允许分析发散，但必须收回结果

复杂分析可以并行拆分，但资源、上下文、结果和交付都必须由 Harness 重新收敛。

开放式分析 问题天然不确定

并行发散 子 Agent 独立验证

结果收敛 统一综合与交付

01

Multi-Agent：并行发散

父 Agent 拆问题，子 Agent 独立完成子任务

Split

按分析问题拆任务

多渠道、多指标、多假设可以并行验证。

Isolate

独立线程、超时与 MCP Client

每个子任务独立管理生命周期，避免互相拖累。

Shared

共享 Builder，保持工具一致

父子 Agent 复用同一套基础工具装配。

Merge

流式回传，统一结果

子结果带标识回写父任务，由父 Agent 综合。

Harness 给并行分析划定资源边界，允许发散但不允许失控。

02

Production Delivery：结果收敛

把长任务变成可恢复、可终止、可交付的生产任务

Recover

Checkpoint 持久化

保存主任务状态，失败后从断点继续。

Stop

跨实例终止与执行租约

控制任务生命周期，避免“停不下来”。

Stream / Poll

Budget

压缩上下文与大结果外置

压缩模型请求，OSS 存放超大结果，保留追溯能力。

Context Budget

Fence

Output Fence 后再交付

飞书 / Web 输出前检查最终可访问性与交付质量。

Output Fence

Harness 负责把资源、上下文、状态和最终产物重新收回到可控边界。

本页结论

Agent 可以自由探索，Harness 必须负责隔离资源、恢复状态、控制预算，并完成最终交付。



AI 数据分析-QNova

QNova Harness：最小数据暴露，让分析能力在企业边界内运行

模型只获得完成分析所需的最小信息，原始数据留在企业执行环境中

安全原则：让 AI 做分析，而不是直接“拿数据” | AI 的认知、代码、执行、结果和权限分别受到控制；数据分析能力可用，但原始数据不直接暴露给模型。

01 · AI 最小可见面

先限制“看什么”，再允许“怎么分析”

READ

Meta

只读取元数据

表、字段、类型、描述等信息用于建立数据认知。

Sample

只提供少量样例

帮助模型理解数据形态，不形成完整原始数据暴露。

Code

分析动作全部代码化

AI 生成分析代码，而不是直接获取整表数据。

Context

原始数据保持隔离

模型通过分析结果获得新信息，不直接接触完整数据集。

02 · 企业内分析执行面

把真正的数据访问和计算放回企业内部

EXECUTE

AI 生成分析代码

SQL

Python

统计分析

数据清洗

模型负责提出分析动作，数据本身不进入模型上下文。

SANDBOX

企业内部 Sandbox Runtime

代码运行、数据访问与计算全部发生在企业内

DATA IN

RESULT

中间结果直接落内部飞书

分析结果作为后续推理输入；AI 读取结果继续分析，不获取完整原始数据。

分析结果 → 下一轮分析决策 · 原始数据始终留在企业执行环境

03 · 治理与权限控制

把“能看到多少”与企业现有权限体系对齐

GOVERN

01

字段级可见性控制

沿用企业既有的数据可见性规则，QNova 不另起权限体系。

02

数据条目数控制

限制单次分析可见规模，避免形成完整数据集暴露。

03

会话级数据传输审计

记录分析会话中的数据传输路径和行为，保证过程可追踪。

04

OA 流程打通

QNova 数据分析权限对齐企业已有 OA 审批和授权流程。

VISIBLE

AI 看到什么

Meta + 少量 Sample + 中间结果

EXECUTE

数据在哪里算

企业内部 Sandbox

CONTROL

谁能看到多少

字段、条目数、企业权限

AUDIT

怎么追溯

会话级数据传输审计

本页结论

QNova 把数据安全从“限制模型能力”转成 **最小可见、企业内执行、结果回流、权限审计的完整 Harness 机制。**

05 总结



总结

企业级 Harness：三个场景，三种核心控制对象

研发、业务 Workflow、数据分析分别面对不同的失控点，因此形成不同的 Harness 设计

共同目标

让 AI 真正进入真实生产环境

Harness 的价值，不是让模型更聪明，而是把模型能力放进一个可执行、可验证、可治理的生产环境。

三个案例的差异可以浓缩成三个控制对象：**执行、流程、数据**。

01 天弦 Harness

AI 研发 / Coding / 工程交付

控制执行

核心对象 工程动作

最大问题：代码生成了，研发任务真的完成了吗？

● 真实研发环境

编译 / 部署 / 日志 / 测试

● CLI 工程化

参数标准化 / 统一错误码

● 环境自动化

环境拉取 / 热部署 / Sandbox

● 工程证据

编译 / Commit / 部署 / 运行

核心闭环

执行 → 验证 → 修复 → 再执行

02 Qmoss Harness

企业业务 Workflow / Agent 工作流

控制流程

核心对象 业务状态与流程

最大问题：AI 能决策，但会不会走偏、死循环，最终不收敛？

● Context

当前 Workflow 状态驱动决策

● Decision

决策受用户 Workflow 约束

● Action

Tool 成功后重新确认业务状态

● Convergence

Success / Failed / Human

核心闭环

状态 → 决策 → 动作 → 回写 → 收敛

03 QNova Harness

企业数据分析 Agent

控制数据

核心对象 分析任务与企业数据

最大问题：AI 能完成分析，同时不越过企业数据边界吗？

● Scene

业务知识 / 数据 / Tools / Model

● Runtime

长任务 / Checkpoint / Multi-Agent

● 最小数据暴露

Meta / Sample / 中间结果

● 治理闭环

字段 / 条目数 / 审计 / OA 权限

核心闭环

最小认知 → 企业内分析 → 结果回流 → 权限审计

最终收束

天弦

控制执行：让 AI 真正做完工程任务

Qmoss

控制流程：让 AI 按状态推进并收敛

QNova

控制数据：让 AI 在边界内完成分析



总结

从天弦、Qmoss、QNova 抽象 Harness 第一性原理

三个场景不同，但背后的工程规律高度一致：Harness 负责把模型的不确定性约束在生产系统可以承受的边界内

FIRST PRINCIPLE

Harness 的本质，是把不确定的模型能力放进一个 可控、可观测、可恢复、可收敛的生产环境。

模型负责理解、推理与决策；Harness 负责边界、状态、执行、证据和终态。
真正被工程化的不是“Prompt”，而是模型所处的整个运行环境。

三个实践给出的共同证据

FROM CASES

01

天弦：模型不能替代工程闭环

代码生成之后，还必须进入真实环境、编译、部署、日志、测试与证据验证。

02

Qmoss：模型不能拥有流程控制权

决策可以开放，但必须被 Workflow State、依赖关系、终态与收敛条件约束。

03

QNova：模型不能直接拥有数据

分析可以开放，但数据访问必须通过最小暴露、Sandbox、权限与审计控制。

由此抽象出的 6 条 Harness 第一性原理

ENGINEERING PRINCIPLES

01

控制环境，而不是依赖模型自律

安全、权限、流程和执行边界必须沉到 Runtime、Tool、Sandbox 和基础设施里，不能只写在 Prompt 中。

03

推理交给模型，确定性动作交给工程系统

模型负责“下一步做什么”，脚本、CLI、Tool 和 Runtime 负责“动作如何可靠发生”。

05

Agent 可以发散，Runtime 必须收敛

任何 Loop、Workflow、Multi-Agent 都必须具备超时、预算、失败、终止、恢复和人工接管机制。

02

把状态放在模型之外

任务状态、Workflow State、Checkpoint、分析上下文都应由系统持有。

模型读取状态，系统拥有状态。

04

所有动作都必须形成可验证证据

Tool 成功、代码生成、SQL 返回都不是终点；必须通过业务状态、测试结果、运行日志或分析产物证明任务真的推进。

06

按最小必要原则开放能力

只暴露完成任务所需的最小工具、最小数据、最小权限和最小上下文，并让每次扩大边界都可审计。

统一结论

天弦约束 执行 · Qmoss 约束 流程 · QNova 约束 数据 → 最终都在做同一件事：约束 AI 所处的生产环境。

QCon

全球软件开发大会

上海
站

大模型正在重新定义软件

Large Language Model Is Redefining The Software

2026 年 10 月 22-24 日 / 上海建工浦江皇冠假日酒店



参会
咨询

**专题：从 App 到 Agent：字节跳动
Supabase 的 Agent-Native 规模化实践**

专题演讲嘉宾：张国强

字节跳动 / 资深技术专家



**专题：复杂业务 Agent 的持续进化：
快手电商导购的 Harness Loop 实践**

专题演讲嘉宾：包磊

快手 / 资深技术专家



**专题：从 Harness 到 Loop：阿福 Agent
小队如何处理持续涌入的线上 Badcase**

专题演讲嘉宾：肖汉松

蚂蚁集团 / 阿福 Harness 架构组负责人



**专题：AI Coding 在大型客户端工程中的
落地实践——从通才模型到专才 Agent**

专题演讲嘉宾：杨凯

小红书 / 客户端基础技术负责人



**专题：AI 时代的一人公司：从 Idea 到
Growth 的完整 workflow**

专题演讲嘉宾：casa

独立开发



**专题：Agent 驱动的全平台 GUI 持续
验证实践**

专题演讲嘉宾：王政达

微软 / SENIOR SOFTWARE ENGINEER



THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会